SOP-001 · Alta de un cliente en el CRM de Comunidades V2

Versión: 1.2 · **Fecha:** 1 de octubre de 2026 · **Responsable:** Taquion (Marketing) · **Cliente de referencia:** Cortado en Jarrito (verificado en producción)

> **Para qué sirve este procedimiento**
> Cada cliente de Taquion que tenga una comunidad entra al mismo CRM. Este SOP explica cómo darlo de alta, cómo dejar su flow de ManyChat listo para enviar los datos al CRM, y cómo comprobar con una prueba real que todo llega a la pestaña **Data_Cruda**. Tiempo estimado: 45 a 60 minutos.

Persona escribe por WhatsApp › **Flow de ManyChat** bienvenida + "Me sumo" › **POST con secreto** nodo final del flow › **CRM (Supabase)** valida y guarda › **Data_Cruda** se ve el evento

0 Antes de empezar

☐ La cuenta de ManyChat del cliente tiene **WhatsApp conectado** (Configuración → WhatsApp → "Estado del número: Conectado") y **saldo en el monedero** (si no, los mensajes no salen).

☐ Tenés permiso de **editor de automatizaciones** en esa cuenta y acceso al proyecto **crm-comunidades** de Supabase (SQL Editor).

☐ Tenés un **gestor de contraseñas** para guardar el secreto del cliente.

☐ Tenés un **teléfono con WhatsApp para la prueba** (el tuyo, no el de un cliente o de un seguidor).

☐ El cliente ya tiene su flow de comunidad armado en ManyChat (disparadores por palabra, mensaje con botón "Me sumo"). Si no, armarlo primero.

1 Reunir los datos del cliente

Dato	Cómo se obtiene / regla	Ejemplo (Cortado en Jarrito)
Nombre	Como se quiere ver en el selector del panel.	Cortado en Jarrito
Slug	Identificador corto: solo minúsculas, números y guiones. No se cambia después.	`cortado-en-jarrito`
ID de cuenta de ManyChat	Es lo que aparece en la dirección web al estar dentro de la cuenta: `app.manychat.com/fb…/…`	`fb5616665`
Comunidades	Una por rama del flow. Se escriben **exactamente** como las va a escribir el flow en el campo de localidad: mismas mayúsculas y tildes, sin emojis ni espacios sobrantes.	Boedo · Palermo · Villa Devoto
Campo de localidad	El campo personalizado de ManyChat que cada rama completa con el nombre de la comunidad.	`Localidad_CABA`
Número de WhatsApp del cliente	Configuración → WhatsApp → "Número conectado".	+54 9 11 2753-5622

2 Crear el secreto del webhook Quien da el alta

El secreto es la "llave" que prueba que un POST viene del flow del cliente. Cada cliente tiene el suyo. Generalo en PowerShell:

```
$b = New-Object byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
-join ($b | ForEach-Object { $_.ToString('x2') })
```

Copiá el resultado (64 caracteres) y **guardalo en el gestor de contraseñas** con el nombre "CRM · secreto webhook · <cliente>".

> **Reglas del secreto**
> Nunca lo pegues en un chat, un mail, un documento compartido ni una captura de pantalla. El CRM no lo guarda: solo guarda una "huella" (hash) que sirve para comprobarlo. Si se filtra, hay que rotarlo (ver el cierre).

3 Dar de alta el cliente y sus comunidades en Supabase Quien da el alta

1. Abrí el **SQL Editor** del proyecto: `supabase.com/dashboard/project/zyyxsammrryhfrgfwlxr/sql/new`
2. Pegá este bloque, reemplazando los textos en MAYÚSCULAS, y apretá **Run**:

```
insert into crm_comunidades_v2.cliente
  (nombre, slug, manychat_account_id, webhook_secret_hash)
values ('NOMBRE DEL CLIENTE', 'slug-del-cliente', 'fbXXXXXXXXX',
        encode(sha256('PEGAR-AQUI-EL-SECRETO'::bytea), 'hex'));

insert into crm_comunidades_v2.comunidad (cliente_id, nombre)
select c.id, n
from crm_comunidades_v2.cliente c,
     unnest(array['Comunidad 1', 'Comunidad 2']) as n
where c.slug = 'slug-del-cliente';

select c.slug, m.nombre
from crm_comunidades_v2.comunidad m
join crm_comunidades_v2.cliente c on c.id = m.cliente_id
where c.slug = 'slug-del-cliente';
```

3. El resultado final tiene que listar una fila por comunidad.
4. **Borrá el snippet guardado:** Supabase conserva las consultas en su historial y esa consulta contiene el secreto. En el panel izquierdo del SQL Editor, eliminá el snippet.
5. Abrí el panel **crm-comunidades-v2.vercel.app**: el cliente nuevo tiene que aparecer en el selector de cliente.

4 Preparar el flow en ManyChat Operador de ManyChat

Trabajá siempre sobre el flow de comunidad del cliente. Los cambios quedan en borrador hasta que se apriete **Actualizar**.

4.1 Cambios que tiene que tener el flow

Parte del flow	Cómo tiene que quedar	Por qué
Disparadores	Uno por comunidad: "El usuario envía un mensaje · **el mensaje contiene** <palabra del barrio>".	Así se activa aunque la persona escriba la frase distinta.

Condiciones	Comparar el campo de localidad con el operador **"Contiene"** y **solo la palabra** (por ejemplo `Boedo`).	Con "Es" y la frase completa (sobre todo si termina en un emoji, como 🟤) cualquier mensaje escrito a mano no coincide y cae en la última rama.
Cada rama	Mensaje de bienvenida con botón "Me sumo" (enlace al grupo) → acciones: añadir la etiqueta del barrio, **establecer el campo de localidad con el nombre exacto de la comunidad** (igual al del paso 1) y establecer `Estado_Comunidad` = "Clic en grupo".	El POST usa ese campo como comunidad. Si no coincide con el CRM, el CRM lo rechaza.
Nodo final	Todas las ramas convergen en un único nodo de acciones. Ahí van, en este orden: (a) planilla de Google (respaldo), (b) **Solicitud externa (POST al CRM)**, (c) evento a la API de conversiones de Meta, si existe, con canal **WhatsApp**.	El POST tiene que salir **después** de las acciones de rama, que son las que dejan la localidad limpia.
Disparador por etiqueta	**No usar** "Se produce un evento de contactos: etiqueta aplicada".	El flow aplica etiquetas: se dispara a sí mismo en bucle.

4.2 Configurar la "Solicitud externa" (POST)

En el nodo final: **+ Acción** → **Automatización** → **Realizar solicitud externa**, y abrirla.

Campo	Valor
Tipo de solicitud	`POST`
URL	`https://zyyxsammrryhfrgfwlxr.supabase.co/functions/v1/manychat-webhook` (sin espacios al final)
Encabezado 1	Key `x-cliente` · Value: el **slug** del cliente
Encabezado 2	Key `x-webhook-secret` · Value: el **secreto** del paso 2 (pegarlo desde el gestor)
Cuerpo	`{"community_id": "[chip del campo de localidad]", "contact": [chip Full Contact Data]}`

- Los dos valores del cuerpo se insertan como **chips** con los botones "{+} Añadir un Campo" (elegir el campo de localidad del cliente) y "+ Añadir Full Contact Data". No se escriben como texto.
- El chip de Full Contact Data va **sin comillas**, y el cuerpo termina con `}`.
- Es normal que la vista previa diga "JSON no válido / Las variables no están definidas" si no hay un contacto de prueba elegido.
- Apretá **Guardar** en la solicitud y después en el nodo.

> **Flows de Instagram**
> El mismo POST sirve para Instagram: se arma igual (mismos encabezados y mismo cuerpo, con el chip de Full Contact Data). Un contacto de Instagram no tiene teléfono: el CRM lo identifica por su identificador de Instagram y lo guarda como `ig:…`. En el panel se distingue con la etiqueta **Instagram**. La misma persona en WhatsApp y en Instagram cuenta como dos.

4.3 Planilla de Google (respaldo)

En la acción de Google Sheets, el mapeo recomendado es: Nombre ← *Nombre completo*, Teléfono ← *WhatsApp ID*, Localidad ← el campo de localidad, Estado ← `Estado_Comunidad`, y una columna por etiqueta ← la etiqueta del mismo nombre. No existe un campo de fecha en ManyChat para mapear: la fecha del CRM es la hora en que llega el POST.

4.4 Publicar

Apretá **Actualizar** (arriba a la derecha). Se publicó cuando el botón queda gris y desaparece el punto naranja junto a la etiqueta **LIVE**. Hasta entonces el flow en vivo sigue siendo el anterior.

5 Prueba final y verificación en Data_Cruda

Operador + quien da el alta

> **Antes de probar**
> Usá solo tu teléfono de prueba. El CRM atribuye cada persona a su **primer contacto**: si tu número ya se usó antes en ese cliente, la prueba no cambia su comunidad. En ese caso, limpiá la prueba anterior (paso 7).

1. Desde el celular de prueba, escribile al **número de WhatsApp del cliente** una frase que contenga la palabra de una comunidad. Ejemplo: `Hola, quiero unirme a la comunidad de Cortado en Jarrito Boedo`
2. Tiene que llegar el **mensaje de bienvenida con el botón "Me sumo"**. Tocalo: se abre el enlace del grupo y llega el mensaje de términos y condiciones. Ahí corrió el nodo final.
3. Abrí **crm-comunidades-v2.vercel.app**, elegí el cliente en el selector y entrá a la pestaña **Data_Cruda**. Tocá **Actualizar** (o marcá "Actualizar cada 30 s").
4. Tiene que aparecer una fila nueva. Comprobá cada columna:

Columna	Qué tiene que decir	Si no coincide
Recibido	La hora de la prueba, con "hace unos segundos".	No llegó: ver el paso 6.
Contacto	El teléfono con código de país, **solo dígitos** (sin "+"), y la etiqueta **WhatsApp**. Si la prueba fue por Instagram: la etiqueta **Instagram** y el identificador `ig:…` (o el @usuario).	Si falta, el contacto no trae teléfono ni identificador de Instagram.
Nombre	El nombre de usuario de WhatsApp de la persona, tal como lo envía ManyChat. Puede ser un alias, un emoji o un punto (**no es un dato verificado**).	Un punto o vacío no es un error del CRM.
Comunidad asignada	La comunidad que escribiste en la prueba.	Otra comunidad: la condición del flow usa "Es" (paso 4.1).
Texto recibido	El nombre limpio de la comunidad (por ejemplo `Boedo`), no la frase completa.	Si llega la frase, el POST está antes de las acciones de rama.
Detalle → Ver JSON	El contenido completo que envió ManyChat.	—

Como verificación extra, en **Overview** la comunidad tiene que sumar **1 clic único**.

> **La prueba pasó si…**
> aparece la fila en Data_Cruda con el teléfono, la comunidad correcta y el texto recibido limpio. Después, repetí con una segunda comunidad para confirmar que cada rama funciona (con otro número, o limpiando antes el anterior).

6 Si algo falla

Síntoma	Causa probable	Qué hacer

No llega el mensaje de bienvenida	El texto no contiene ninguna palabra de disparador; el flow no está publicado; el monedero no tiene saldo.	Revisar los disparadores; confirmar que no hay punto naranja junto a LIVE; ver Configuración → Registros ("insufficient funds in your Wallet") y recargar el monedero.
Llegó la bienvenida pero no hay fila en Data_Cruda	La persona no tocó "Me sumo" (el nodo final no corrió), o el POST fue rechazado.	Tocar "Me sumo". Si ya lo tocó, abrir Data_Cruda → filtro **Rechazados**: ahí aparece el código y el motivo. Compararlos con los códigos de abajo.
Código **401**	Secreto o slug incorrectos (o cliente inactivo).	Volver a pegar el secreto desde el gestor, sin espacios; revisar que `x-cliente` sea el slug exacto.
Código **404** "Comunidad no encontrada"	El nombre que llega en `community_id` no existe en ese cliente.	Igualar el nombre del flow con el del CRM (mayúsculas, tildes, espacios).
Código **400**	No llegó ni teléfono (WhatsApp) ni `ig_id` (Instagram), o falta `community_id`.	El motivo en Data_Cruda lista los campos del contacto que sí llegaron. Revisar que el chip del cuerpo no esté vacío.
Código **501**	El cliente no tiene secreto cargado.	Repetir el paso 3.
Aparece en la comunidad equivocada (siempre la última)	Condiciones con "Es" y frase exacta (o con emoji).	Cambiar a "Contiene" con la palabra sola (paso 4.1).
Hay fila en la planilla pero no en el CRM	El POST no está en el nodo final, o los cambios no se publicaron.	Revisar el nodo final y apretar Actualizar.
La segunda prueba no cambia la comunidad	Atribución al primer contacto del mismo número.	Limpiar la prueba anterior (paso 7).

Ver los rechazos en Data_Cruda
En Data_Cruda, el filtro **Rechazados** lista cada POST que el CRM rechazó, con el código (401, 400, 404, 501) y el motivo en palabras: es la forma más rápida de diagnosticar. **Límite:** no se registran los intentos que no identifican a un cliente (encabezados ausentes o `x-cliente` con un slug que no existe). Para esos, mirar los Logs de la función en Supabase.

7 Limpiar los datos de prueba

Quien da el alta

Solo con tu teléfono de prueba, en el SQL Editor (reemplazá el slug y el número, sin "+"). **Revisá dos veces el número antes de apretar Run: borra definitivamente.**

```
delete from crm_comunidades_v2.evento_crudo
 where telefono_id = 'NUMERO-DE-PRUEBA'
   and cliente_id = (select id from crm_comunidades_v2.cliente where slug = 'slug-del-cliente');
delete from crm_comunidades_v2.usuario_estado_historial
 where telefono_id = 'NUMERO-DE-PRUEBA'
   and cliente_id = (select id from crm_comunidades_v2.cliente where slug = 'slug-del-cliente');
delete from crm_comunidades_v2.usuario_curado
 where telefono_id = 'NUMERO-DE-PRUEBA'
   and cliente_id = (select id from crm_comunidades_v2.cliente where slug = 'slug-del-cliente');
```

Las filas de prueba de la planilla de Google se borran a mano.

8 Cierre: checklist final

- ☐ Cliente visible en el selector del panel, con todas sus comunidades.
- ☐ Secreto guardado en el gestor de contraseñas y snippet de Supabase borrado.
- ☐ Flow con disparadores por palabra, condiciones “Contiene”, y la localidad de cada rama igual al nombre en el CRM.
- ☐ POST en el nodo final, con los dos encabezados y el cuerpo con chips, y flow **publicado** (sin punto naranja).
- ☐ Evento a la API de conversiones (si existe) con canal WhatsApp.
- ☐ Prueba real hecha: la fila apareció en Data_Cruda con comunidad y texto correctos.
- ☐ Datos de prueba limpiados en el CRM y en la planilla.

Cosas a tener presentes

- **El panel todavía no tiene login.** Quien tenga la dirección puede ver teléfonos y nombres: no compartirla fuera del equipo, y no abrir el flow al público hasta que el login esté activo.
- **Rotar el secreto** si se filtró, o antes de abrir el flow al público: generar uno nuevo (paso 2), actualizar el hash del cliente (`update crm_comunidades_v2.cliente set webhook_secret_hash = encode(sha256('NUEVO'::bytea),'hex') where slug = '…'`) y cambiar el valor del encabezado `x-webhook-secret` en ManyChat.
- **Datos de Meta Ads** (inversión, clics, conversaciones) se conectan aparte con Windsor.ai. No forman parte de este SOP.